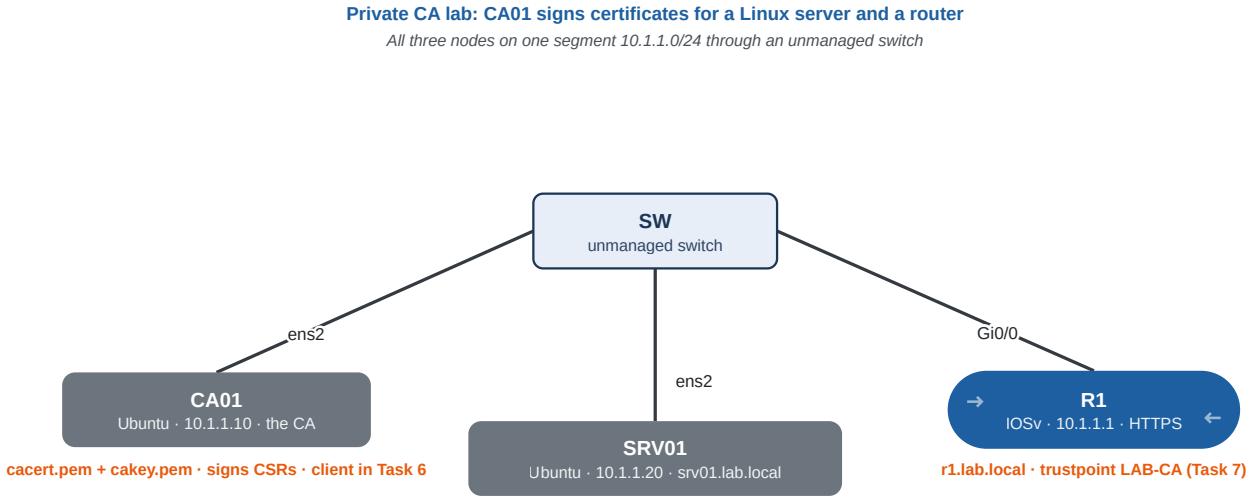
Certificates Lab: Private CA, CSR Signing and Trust

Lab Objective:

The focus of this lab is the basic certificate workflow from your Chapter 8 notes: build a private CA with OpenSSL, create a CSR on a server, sign it on the CA, use the certificate for HTTPS, and make a client trust the CA. In the last task a Cisco router goes through the same workflow.

Lab Topology:

The lab topology is illustrated below:



IMPORTANT NOTE

CA01 and SRV01 are Ubuntu nodes, R1 is IOSv, SW is an unmanaged switch. Tasks 1 to 6 need only the two Ubuntu nodes; R1 is used in Task 7.

The whole lab is one idea, done twice: **key + CSR → the CA signs it → the server uses the certificate → clients that trust the CA trust the server.**

Moving files between the nodes: just copy and paste. Show the file with `cat` on one node and paste it into `nano` on the other. Certificates and CSRs are plain text.

Task 1

Give CA01 10.1.1.10/24, SRV01 10.1.1.20/24 and R1 10.1.1.1/24. On CA01, add the names `srv01.lab.local` and `r1.lab.local` to `/etc/hosts`. Verify that CA01 can ping SRV01 by name.

Task 2

On CA01, build the private CA with your notes (Steps 1 to 5). Before creating the CA certificate, set `[ CA_default ]` in `/etc/ssl/openssl.cnf` so that SANs from a CSR end up in the certificate and the CA

doesn't care about Country, State or Organisation. Show the CA certificate's subject, its basic constraints and its SHA-1 fingerprint.

Task 3

On SRV01, create a private key and a CSR with your notes. The certificate must work for the name `srv01.lab.local` and the IP `10.1.1.20`. Check that the SAN is in the CSR.

Task 4

Copy the CSR to CA01 and sign it. Show what changed in the CA's `index.txt` and `serial`, verify the new certificate against the CA, and copy the certificate back to SRV01.

Task 5

Run a test HTTPS server on SRV01 port 443 with the new certificate and key.

Task 6

From CA01, connect to `https://srv01.lab.local` with curl. Make CA01 trust your CA, then connect again. Also connect by IP, and by a name that isn't in the certificate, and explain the results.

Task 7

Give R1 a certificate from your CA and make its HTTPS server use it:

- Set R1's clock to CA01's time
- Trustpoint **LAB-CA** with copy-paste enrollment, subject `CN=r1.lab.local`, no revocation check
- Install the CA certificate on R1, checking the fingerprint from Task 2
- Create R1's CSR, sign it on CA01, import the certificate into R1

Connect to `https://r1.lab.local` from CA01 with curl.

Lab Validation:

The outputs below are trimmed. Dates, serial numbers and fingerprints will differ in your lab.

Task 1

```
cisco@ca01:~$ sudo ip addr add 10.1.1.10/24 dev ens2
cisco@ca01:~$ sudo ip link set ens2 up
cisco@srv01:~$ sudo ip addr add 10.1.1.20/24 dev ens2
cisco@srv01:~$ sudo ip link set ens2 up
```

```
R1(config)#interface GigabitEthernet0/0
R1(config-if)#ip address 10.1.1.1 255.255.255.0
R1(config-if)#no shutdown
```

```
cisco@ca01:~$ echo '10.1.1.20 srv01.lab.local' | sudo tee -a /etc/hosts
cisco@ca01:~$ echo '10.1.1.1 r1.lab.local' | sudo tee -a /etc/hosts
```

```
cisco@ca01:~$ ping -c 2 srv01.lab.local
64 bytes from srv01.lab.local (10.1.1.20): icmp_seq=1 ttl=64 time=0.52 ms
```

Use a netplan file, as in your notes, if the addresses must survive a reboot.

Task 2

Steps 1 and 2 from your notes:

```
cisco@ca01:~$ sudo mkdir /etc/ssl/CA
cisco@ca01:~$ sudo mkdir /etc/ssl/newcerts
cisco@ca01:~$ sudo sh -c "echo '01' > /etc/ssl/CA/serial"
cisco@ca01:~$ sudo touch /etc/ssl/CA/index.txt
```

Step 3: edit `/etc/ssl/openssl.cnf` (`sudo nano /etc/ssl/openssl.cnf` , Ctrl+W `CA_default`). Make the section look like this, and make sure each key appears only **once** (if a key is repeated, the last one wins):

```
[ CA_default ]
dir             = /etc/ssl
database        = $dir/CA/index.txt
serial          = $dir/CA/serial
new_certs_dir   = $dir/newcerts
certificate     = $dir/certs/cacert.pem
private_key     = $dir/private/cakey.pem
x509_extensions = usr_cert
copy_extensions = copy
policy          = policy_anything
default_days    = 365
default_md      = sha256
```

Line	Why
`copy_extensions = copy`	Without it, `openssl ca` **drops the SAN** from the CSR, and clients reject the certificate
`policy = policy_anything`	The default `policy_match` refuses CSRs whose Country, State or Organisation differ from the CA's. A router's CSR doesn't have them. `policy_anything` only needs a CN

Step 4: create the CA certificate. Choose a passphrase for the CA key, and type `.` at every prompt except the Common Name, which is just the CA's name:

```
cisco@ca01:~$ openssl req -new -x509 -extensions v3_ca -keyout cakey.pem -out cacert.pem -days 3650
Enter PEM pass phrase:
Verifying - Enter PEM pass phrase:
Country Name (2 letter code) [AU]:.
State or Province Name (full name) [Some-State]:.
Locality Name (eg, city) []:.
Organization Name (eg, company) [Internet Widgits Pty Ltd]:.
Organizational Unit Name (eg, section) []:.
Common Name (e.g. server FQDN or YOUR name) []:CA
Email Address []:.
```

Step 5: move the files:

```
cisco@ca01:~$ sudo mv cakey.pem /etc/ssl/private/
cisco@ca01:~$ sudo mv cacert.pem /etc/ssl/certs/
```

```
cisco@ca01:~$ openssl x509 -in /etc/ssl/certs/cacert.pem -noout -subject -ext basicConstraints -
fingerprint -sha1
```

```
subject=CN = CA
X509v3 Basic Constraints: critical
    CA:TRUE
SHA1 Fingerprint=3A:9F:51:...:C4
```

CA:TRUE means this certificate may sign other certificates. Write the fingerprint down for Task 7.

Task 3

Your notes' Steps 1, 1b and 2, on SRV01. First the key and the passphrase-free copy for the service:

```
cisco@srv01:~$ openssl genrsa -aes256 -out server.key 2048
cisco@srv01:~$ openssl rsa -in server.key -out server.key.insecure
cisco@srv01:~$ mv server.key server.key.secure
cisco@srv01:~$ mv server.key.insecure server.key
```

Then the CSR. Type `.` at every prompt except the Common Name, `srv01.lab.local`:

```
cisco@srv01:~$ openssl req -new -key server.key -out server.csr -addext
"subjectAltName=DNS:srv01.lab.local,IP:10.1.1.20"
cisco@srv01:~$ openssl req -in server.csr -noout -text | grep -A1 "Subject Alternative"
            X509v3 Subject Alternative Name:
                DNS:srv01.lab.local, IP Address:10.1.1.20
```

The SAN lists every name and IP clients will use to reach this server. The private key never leaves SRV01: only the CSR goes to the CA.

Task 4

Copy the CSR to CA01:

```
cisco@srv01:~$ cat server.csr
-----BEGIN CERTIFICATE REQUEST-----
MIICnDCCAYQCAQAwGjEYMBYGA1UEAwwPc3J2MDEubGFiLmxvY2Fs...
-----END CERTIFICATE REQUEST-----
cisco@ca01:~$ nano srv01.csr
```

Paste everything from `-----BEGIN` to `-----END`, save with Ctrl+O, exit with Ctrl+X. Sign it, typing the **CA's** passphrase:

```
cisco@ca01:~$ sudo openssl ca -in srv01.csr -out srv01.crt -config /etc/ssl/openssl.cnf
Enter pass phrase for /etc/ssl/private/cakey.pem:
Check that the request matches the signature
Signature ok
Certificate Details:
        Serial Number: 1 (0x1)
        Subject:
            commonName                = srv01.lab.local
        X509v3 extensions:
            X509v3 Subject Alternative Name:
                DNS:srv01.lab.local, IP Address:10.1.1.20
Certificate is to be certified until Nov  7 11:20:31 2027 GMT (365 days)
Sign the certificate? [y/n]:y
1 out of 1 certificate requests certified, commit? [y/n]y
Data Base Updated
```

The SAN is in the details: `copy_extensions` worked.

```
cisco@ca01:~$ cat /etc/ssl/CA/index.txt
V   271107112031Z       01  unknown /CN=srv01.lab.local
cisco@ca01:~$ cat /etc/ssl/CA/serial
02
cisco@ca01:~$ openssl verify -CAfile /etc/ssl/certs/cacert.pem srv01.crt
srv01.crt: OK
```

`index.txt` records the certificate (`V` = valid, expiry date, serial 01, subject). `serial` moved on to 02 for the next one.

`srv01.crt` starts with a long text dump. `openssl x509` prints just the PEM part, which is what you copy back:

```
cisco@ca01:~$ openssl x509 -in srv01.crt
-----BEGIN CERTIFICATE-----
MIIDLTCCAhWgAwIBAgIBATANBgkqhkiG9w0BAQsFADANMQswCQYDVQQDDAJDQTAe...
-----END CERTIFICATE-----
cisco@srv01:~$ nano server.crt
```

Task 5

```
cisco@srv01:~$ sudo openssl s_server -accept 443 -cert server.crt -key server.key -www
Using default temp DH parameters
ACCEPT
```

It keeps running. Leave it open. `-www` answers every HTTPS request with a small status page. `sudo` is only needed because port 443 is below 1024.

Task 6

```
cisco@ca01:~$ curl https://srv01.lab.local
curl: (60) SSL certificate problem: unable to get local issuer certificate
```

The server's certificate is fine, but CA01's **trust store** doesn't know your CA. Install the CA certificate the way your notes describe:

```
cisco@ca01:~$ sudo cp /etc/ssl/certs/cacert.pem /usr/local/share/ca-certificates/my-ca.crt
cisco@ca01:~$ sudo update-ca-certificates
1 added, 0 removed; done.
cisco@ca01:~$ curl -s https://srv01.lab.local | head -2
<HTML><BODY BGCOLOR="#ffffff">
<pre>
```

Same server and same certificate. The only change is that the client now trusts the **issuer**. The file name must end in `.crt`.

```
cisco@ca01:~$ curl -s https://10.1.1.20 | head -1
<HTML><BODY BGCOLOR="#ffffff">
cisco@ca01:~$ echo '10.1.1.20 www.lab.local' | sudo tee -a /etc/hosts
cisco@ca01:~$ curl https://www.lab.local
curl: (60) SSL: no alternative certificate subject name matches target host name 'www.lab.local'
```

Connect to	Result	Why
`srv01.lab.local`	Works	In the SAN
`10.1.1.20`	Works	The IP is in the SAN too
`www.lab.local`	Fails	Same server, same certificate, but that name isn't in the SAN

The client compares the name **it connected to** with the names **in the certificate**. If none match, it refuses, even though the CA is trusted.

Task 7

Certificates have a start date, and IOSv boots with a random clock. If R1 thinks it's last year, your certificate is "not yet valid". Copy CA01's UTC time to R1:

```
cisco@ca01:~$ date -u
Sat Nov  7 11:35:02 UTC 2026
R1#clock set 11:35:30 7 Nov 2026
```

Key pair and trustpoint:

```
R1(config)#hostname R1
R1(config)#ip domain name lab.local
R1(config)#crypto key generate rsa modulus 2048 label R1-KEY
R1(config)#crypto pki trustpoint LAB-CA
R1(ca-trustpoint)#enrollment terminal pem
R1(ca-trustpoint)#subject-name CN=r1.lab.local
R1(ca-trustpoint)#rsakeypair R1-KEY
R1(ca-trustpoint)#revocation-check none
R1(ca-trustpoint)#exit
```

Command	Meaning
`crypto pki trustpoint LAB-CA`	One CA the router trusts, plus how it gets certificates from it
`enrollment terminal pem`	Copy-paste enrollment, the same as you did for SRV01
`rsakeypair R1-KEY`	Which key goes into R1's certificate
`revocation-check none`	Don't look for a CRL (we don't publish one)

Install the CA certificate: the router version of `update-ca-certificates`. Paste the content of `/etc/ssl/certs/cacert.pem`, then type `quit` on a line of its own:

```
R1(config)#crypto pki authenticate LAB-CA
Enter the base 64 encoded CA certificate.
End with a blank line or the word "quit" on a line by itself
-----BEGIN CERTIFICATE-----
...
-----END CERTIFICATE-----
quit
Certificate has the following attributes:
      Fingerprint SHA1: 3A9F51...C4
% Do you accept this certificate? [yes/no]: yes
Trustpoint CA certificate accepted.
```

Compare the SHA-1 fingerprint with Task 2 **before** you say yes. That's how you know it's really your CA.

R1's CSR:

```
R1(config)#crypto pki enroll LAB-CA
% Include the router serial number in the subject name? [yes/no]: no
% Include an IP address in the subject name? [no]: no
Display Certificate Request to terminal? [yes/no]: yes
-----BEGIN CERTIFICATE REQUEST-----
MIICsTCCAZkCAQAwNjEVMBMGA1UEAxMMcjEubGFiLmxvY2Fs...
-----END CERTIFICATE REQUEST-----
```

On CA01: paste it into `r1.csr`, sign it, print the PEM part:

```
cisco@ca01:~$ nano r1.csr
cisco@ca01:~$ sudo openssl ca -in r1.csr -out r1.crt -config /etc/ssl/openssl.cnf
cisco@ca01:~$ openssl x509 -in r1.crt
```

Import it on R1. Paste the PEM, then `quit`:

```
R1(config)#crypto pki import LAB-CA certificate
% Router Certificate successfully imported
R1#show crypto pki certificates
Certificate
  Status: Available
  Issuer:
    cn=CA
  Subject:
    Name: R1.lab.local
    hostname=R1.lab.local
    cn=r1.lab.local
  Associated Trustpoints: LAB-CA
CA Certificate
  Status: Available
  Subject:
    cn=CA
  Associated Trustpoints: LAB-CA
```

R1 now holds the **CA certificate** (which it trusts) and **its own certificate** (which proves who it is). IOS added `hostname=R1.lab.local` to the subject by itself. That's another reason for `policy_anything`.

Use it for HTTPS:

```
R1(config)#ip http secure-server
R1(config)#ip http secure-trustpoint LAB-CA
cisco@ca01:~$ curl -sI https://r1.lab.local
HTTP/1.1 401 Unauthorized
```

401 is R1's web server asking for a login, so the **TLS part worked**: curl trusted R1's certificate because CA01 already trusts your CA since Task 6. Without `secure-trustpoint`, IOS would present a self-signed certificate and curl would refuse it.